# Recriar um quarteirão de favela carioca em 3D com alta fidelidade

Levantamento técnico de caminhos, softwares e evidência empírica. Rio de Janeiro, 12 de setembro de 2026. Preparado para o projeto *Machine Vision from the Global South: Developing an AI Model for Vertical Favelas*.

> **Resposta curta.** O caminho de maior fidelidade não é modelar uma favela bonita em Blender nem comprar um kit de assets. É reconstruir a geometria real do lugar por fotogrametria ou LiDAR e, sobre ela, variar agressivamente sol, clima e ângulo de câmera. Existe evidência publicada e medida de que essa escolha vale mais de 22 pontos de mAP contra a alternativa de assets genéricos, e que a variação de iluminação vale outros 10 a 20 pontos. Comprar um kit de favela e caprichar na textura é, literalmente, o braço perdedor do experimento mais próximo do seu.

## 1. O que este levantamento responde

A pergunta original tinha três partes: qual o melhor caminho no Blender para criar cidades com alto realismo, se faz sentido usar outro software como Unity, e se há integração com Claude para validar o processo. A pesquisa mudou a ordem de importância dessas perguntas.

O objetivo declarado é recriar uma quadra de favela com alta fidelidade de textura, realismo e sombras/sol, com qualidade de videogame. Como o projeto em que isso se insere treina um modelo de visão computacional sobre dados sintéticos, existe um critério objetivo de sucesso que não é estético: o modelo treinado nas imagens renderizadas precisa funcionar em imagem real. Esse critério é mensurável, já foi medido por outros, e é ele que organiza o documento.

Se o objetivo fosse apenas uma imagem bonita para publicação ou apresentação, várias conclusões aqui seriam diferentes e mais permissivas. Onde isso acontece, está sinalizado.

## 2. A evidência que decide o desenho

### 2.1 O experimento mais próximo do seu já foi feito e publicado

Liao, Liu, Xu, Wang, Liu, Yang e Li publicaram em 2026 na *Remote Sensing* (18(2):361, DOI 10.3390/rs18020361) exatamente o pipeline que este levantamento recomenda, para detecção de veículos em imagem de drone. A estrutura tem três estágios: fotogrametria oblíqua de UAV reconstrói o modelo 3D real do local, estratégias de renderização diversificadas simulam variação de iluminação e clima, e um algoritmo automatizado gera ground truth a partir de máscaras semânticas de instância.

Stack: Colosseum (sucessor do AirSim) sobre Unreal Engine 5.1, com o plugin UltraDynamicSky para elevação solar, cor da luz, intensidade de sombra e condições de tempo. Câmera de 10 mm de focal, sensor de 6,4 por 5,12 mm, resolução 1920 por 1536. Treino em MMDetection numa única RTX 4090.

O dataset resultante, UAV-SynthScene, tem **4.659 imagens**. Dez rotas de voo, altitude sorteada entre 50 e 300 m, heading entre -10 e 10 graus, pitch entre -45 e -90 graus, roll fixo em zero. Três condições de tempo (sol, nublado, névoa) e três de luz (nascer do sol, meio-dia, pôr do sol).

| Resultado (mAP50, sempre avaliado em conjunto de teste REAL) | Valor |
|---|---|
| DINO treinado só em sintético | **87,9%** |
| DINO treinado em dado real | 87,9% (idêntico) |
| DEIM, sintético deles | **88,7%** |
| DEIM, treinado em SkyScenes | 86,5% |
| DEIM, treinado em UEMM-Air | 72,0% |
| Objetos truncados, DINO, sintético deles | **73,0%** |
| Objetos truncados, DINO, dado real | 56,3% |
| Zero-shot em RGBT-Tiny, DINO, sintético deles | **96,5%** |
| Zero-shot em RGBT-Tiny, DINO, SkyScenes | 76,5% |

### 2.2 As duas ablações que respondem diretamente à sua pergunta

O valor deste paper para você não está no resultado principal, está nas duas ablações. Elas isolam exatamente as duas variáveis sobre as quais você perguntou.

> **Ablação A, remover a variação de iluminação e de ângulo.** Renderizando tudo em condição fixa (meio-dia claro, altitude 150 m, pitch -80 graus, roll zero), a média de mAP50 cai cerca de **10 pontos** no conjunto completo e cerca de **20 pontos** no subconjunto de objetos truncados. O DINO em objetos truncados despenca de 73,0% para 44,2%.

> **Ablação B, trocar a base fotogramétrica por assets 3D genéricos**, mantendo toda a variação de iluminação. A média cai **22,5 pontos de mAP50** (a seção de discussão do paper cita 23,3 pontos, pequena inconsistência interna). O RT-DETR perde 27,5 pontos.

Traduzindo para as suas decisões: a variação de sol e sombra que você intuiu como importante é de fato essencial e vale cerca de 10 a 20 pontos. Mas a origem da geometria vale mais do que o dobro disso. Nenhum esforço de textura, shader ou iluminação compensa modelar a favela errada.

Os autores formulam assim: *"for top-down remote sensing views, structural and textural fidelity is a more critical factor than for ground-level vision, where generic urban layouts can often suffice."* E advogam explicitamente por integrar técnicas de mapeamento 3D de grau topográfico, fotogrametria e LiDAR, dentro do pipeline de dado sintético.

### 2.3 O contraponto que parece contradizer e não contradiz

Há uma linha de literatura que aponta na direção oposta: Tremblay et al. (CVPRW 2018) mostraram que randomização de domínio não fotorrealista bate sintético fotorrealista em duas de três arquiteturas no KITTI, e Barisic et al. (Sim2Air, RA-L 2022) obtiveram ganhos de 17 a 20 pontos percentuais justamente usando texturas irrealistas. Krump e Stütz (Sensors 2023) decompõem o problema em *content gap* (o que está na cena) e *appearance gap* (como é renderizado) e concluem que similaridade de conteúdo domina fotorrealismo.

Isso não contradiz o resultado anterior, converge com ele. Fotogrametria resolve o *content gap*, que é o fator dominante. Randomização agressiva de luz, cor e material ataca o *appearance gap*, que é o fator secundário mas não desprezível. Fazer os dois é precisamente a definição de **randomização estruturada de domínio** (Prakash et al., ICRA 2019), em que as distribuições amostradas derivam da estrutura real do problema em vez de serem uniformes.

> **Princípio operacional que sai daqui:** acerte a geometria com dado real, e depois randomize a aparência com base nas distribuições observadas nas ortofotos reais (cor de reboco, material de telhado, altura de laje, densidade de antena). Isso é um argumento metodológico defensável para a seção de método do artigo, não apenas uma escolha técnica.

### 2.4 Expectativa calibrada, para não prometer o que não se entrega

A literatura de detecção aérea é honesta sobre os limites, e convém registrar antes de qualquer promessa em proposta ou artigo.

- Kiefer, Ott e Zell (ICPR 2022) reportam sintético puro no VisDrone com **1,2 mAP** (EfficientDet) e 2,4 mAP (Faster R-CNN), contra 24,6 e 48,6 do dado real. Colapso quase total em cena urbana densa com objetos minúsculos.
- Rizzoli et al. (SynDrone, ICCVW 2023) mostram que treinar a 20 m de altitude e testar a 80 m derruba o mIoU de 65,1 para 28,1. **Altitude é um eixo de domain shift tão severo quanto clima.** Varrer altitude não é opcional.
- Khose et al. (SkyScenes, ECCV 2024) medem ganho de praticamente zero ao adicionar sintético quando já existe 100% do dado real e um backbone forte (DINOv2). O sintético resolve escassez, não abundância.
- Consolidando Burdorf et al. (2022), Dergachov e Aizatskyi (WDA 2026), SkyScenes e Lee et al. (2024): com pré-treino sintético de qualidade, **5% a 20% do volume de dado real recupera 85% a 95% do desempenho de treino totalmente real**. Planeje anotar algumas centenas de imagens reais. Não prometa substituição total.

Um detalhe técnico fácil de perder e caro: o Unity anota apenas a porção visível do objeto sob oclusão (modal), enquanto o VisDrone anota o objeto inteiro (amodal). Essa incompatibilidade de convenção gera falsos positivos e falsos negativos simultâneos, é invisível numa inspeção visual do dataset, e custa pontos de mAP sem causa aparente. Defina e documente a convenção antes de gerar o primeiro lote.

## 3. De onde vem a geometria da favela

Como a seção anterior estabeleceu que esta é a decisão de maior peso, ela vem antes da escolha de software. Todas as opções abaixo foram verificadas.

### 3.1 A descoberta mais acionável do levantamento: LiDAR e ortofoto 2024 do IPP

> O Instituto Pereira Passos mantém publicamente servida uma **ortofoto verdadeira de toda a cidade do Rio, adquirida no primeiro semestre de 2024, com pixel de 15 cm**, produzida por um voo **que incluiu levantamento LiDAR**. Serviço ArcGIS em `pgeo3.rio.rj.gov.br/arcgis/rest/services/Imagens/Mosaico_2024/ImageServer`, RGB, EPSG:31983 (UTM 23S, SAD69), extensão X de 621.720 a 695.788 e Y de 7.444.172 a 7.485.303, que cobre o Jacarezinho. Copyright Instituto Pereira Passos, PCRJ.

Isso muda o projeto em três frentes. Primeiro, é dado municipal aberto e recente, não dado sensível da PCERJ, o que remove o problema ético e de sigilo que hoje trava a circulação do material de 2022. Segundo, 15 cm de GSD é resolução de sobra para gerar textura e validar geometria. Terceiro, e mais importante, se a nuvem LiDAR do mesmo voo estiver disponível, ela é a melhor fonte possível de morfologia vertical de favela, que é literalmente o objeto do artigo.

O catálogo público da Mapoteca do IPP lista apenas produtos de 1975 a 2019. O acervo restante é obtido por solicitação em `cartografia.ipp@prefeitura.rio`, sujeito a licença de uso, com desconto para estudantes. O servidor expõe mosaicos de 1999 até 2025, mas **nenhum serviço de LiDAR, MDS, MDT ou nuvem de pontos está publicado**. Ou seja: a ortofoto está disponível agora, a nuvem LiDAR precisa ser pedida.

> **Ação de maior alavancagem do projeto inteiro:** escrever para `cartografia.ipp@prefeitura.rio` solicitando a nuvem de pontos LiDAR e o MDS do levantamento de 2024 para o recorte do Jacarezinho, identificando-se como pesquisador de mestrado da FGV EBAPE. Isso deve ser feito antes de qualquer decisão de software, porque a resposta determina o desenho. Não verifiquei disponibilidade real, custo nem termos.

### 3.2 O que você já tem

Os dois arquivos de Jacarezinho de 2022 no Drive (Sudoeste, 145 MB; Nordeste, 248 MB), do voo da PCERJ, continuam sendo um ativo valioso: são a geometria correta de uma favela vertical real, com posição real de cada edificação. Reconstruir isso à mão levaria meses e seria bem menos defensável academicamente.

Duas ressalvas. São dados sensíveis de origem policial, o que restringe publicação e compartilhamento (a nota do projeto já registra que não devem entrar em pacotes). E têm quatro anos, num tecido urbano que muda rápido. Se o LiDAR 2024 do IPP vier, ele é preferível em todos os aspectos: mais novo, mais preciso em vertical, e publicável.

### 3.3 O que está juridicamente bloqueado

> **Google Photorealistic 3D Tiles está fora, apesar de cobrir o Rio.** As Map Tiles API Policies proíbem literalmente *"image analysis, machine interpretation, object detection or identification, geodata extraction or resale, or offline uses"*, e também cache, pré-fetch, armazenamento e criação de obra derivada. A cláusula complementar permite sobrepor objetos 3D próprios aos tiles *"as long as the 3D objects aren't extracted, traced, or otherwise derived by hand or machine"* deles. Treinar um detector sobre esses tiles é exatamente o uso vedado. Não há exceção acadêmica em nenhum dos termos, nem do Google nem da Cesium.

Vale como referência visual em tela e para contexto de skyline em figura, com atribuição obrigatória (logo Cesium ion e logo Google Maps). Não vale como fonte de geometria nem como cena de captura de dataset.

### 3.4 Demais fontes, avaliadas

| Fonte | Serve para | Limite |
|---|---|---|
| **LiDAR IPP 2024** (sob solicitação) | Morfologia vertical real, a melhor opção | Disponibilidade não confirmada |
| **Ortofoto IPP 2024**, 15 cm, pública | Textura, validação, footprints, referência de cor real | É 2,5D, não dá volumetria sozinha |
| Fotogrametria PCERJ 2022 (sua) | Geometria real do Jacarezinho | Sensível, não publicável, desatualizada |
| DATA.RIO, camada "Edificações" | Footprints 2D, escala 1:2.000 | Restituída sobre imagem de **2013**. Muito desatualizada para área informal |
| IBGE, "Favelas e Comunidades Urbanas" (Censo 2022) | Perímetro oficial, contexto estatístico | Polígono de área, sem edificação individual |
| OpenStreetMap (via Blosm) | Traçado de vielas onde mapeado | Cobertura de favela notoriamente incompleta. Ruas do Vidigal aparecem vazias no OSM e no Google Maps |
| Google Photorealistic 3D Tiles | Nada, neste projeto | Proibido contratualmente para detecção de objetos |

### 3.5 Kits de assets: úteis, mas não para a massa construída

Existem e são bons, mas a Ablação B da seção 2.2 diz que usá-los *no lugar* da geometria real custa mais de 22 pontos de mAP. Use-os para props e detalhe de alta frequência sobre a base real, nunca como a base.

| Pack | Preço | Conteúdo | Observação |
|---|---|---|---|
| **KitBash3D "Favelas"** | US$ 195 no site, US$ 199,99 na Fab | 256 modelos, 7,5M polys, 43 materiais PBR. Blender, UE, Houdini, OpenUSD, MaterialX | O melhor do mercado em qualidade, com duas ressalvas sérias. A ficha da Fab avisa: *"These are high poly assets intended for use in cinematics within Unreal Engine 5."* **Não é game-ready**, o que pesa num pipeline de render em lote. E a descrição vende as moradias para *"burgeoning post-apocalyptic urban centers"*, sem menção a Rio ou Brasil: estética de favela global, não morfologia carioca. As layout scenes vêm em todos os formatos exceto Unreal e Unity. Tag "Allows usage with AI" na Fab: **No** |
| **South American Slums - Favela** (Leartes) | US$ 49,99 na Fab, US$ 79,99 na Unity Asset Store | ~531 meshes, 324 materiais, 797 texturas (4K/2K/1K), 4,7 GB | O mais denso. Versão Unity atualizada em 08/04/2026 com suporte a HDRP. **Atenção:** vem com Nanite habilitado e **nenhum LOD**. Como o Blender não tem equivalente a Nanite, esse pack é substancialmente mais pesado num pipeline Blender do que no Unreal para o qual foi feito |
| **Megapack Favela Rio de Janeiro Brazil** (Studio 17) | ~US$ 125 | UE 5.4 | **Baseado especificamente na Rocinha.** O único com referência carioca declarada. Tag AI: **Yes** |
| Stylized Favela Residential Kit (RealityForge) | ~US$ 5 a 19 | 80+ meshes, vertex paint, gerador de cabos em blueprint | Estilizado, não fotorrealista. Tag AI: No |
| Brazilian Favela Modular Pack (Space Studios) | US$ 20 | 186 MB, low-poly mobile | Insuficiente para fotorrealismo |
| Favela Inspired Old Post Apocalyptic Buildings (hknoblauch) | US$ 25 na Superhive/Blender Market, US$ 31,50 na CGTrader | Assets de edificação | Opção barata para props. Estética pós-apocalíptica, não carioca |

> **Atenção de licença que quase ninguém checa.** A EULA da Fab restringe conteúdo marcado **NoAI**: não pode ser usado em datasets para treinar, nem no desenvolvimento de, "Generative AI Programs". O KitBash3D Favelas é NoAI. Porém, a própria definição contratual exclui programas que *"generate tags to classify visual input content"*, o que, em leitura literal, deixa um detector discriminativo fora da restrição. O ponto genuinamente cinzento é **publicar o dataset sintético completo** como artefato aberto em Zenodo ou HuggingFace, o que pode ser lido como redistribuição do conteúdo. Mitigação prática: preferir assets sem tag NoAI, usar Megascans para superfície, e manter rastreabilidade asset por imagem. Isto é leitura de contrato de adesão, não parecer jurídico.

### 3.6 Precedente acadêmico que você deve citar

- **Berwaldt, N. L. P.; Bettker, R. V.; Pozzer, C. T. "Procedural Generation of Favela Layouts on Arbitrary Terrains".** SBGames 2020, 19º Simpósio Brasileiro de Jogos, Recife, pp. 136-144, IEEE, DOI 10.1109/SBGames51465.2020.00027. Trabalho brasileiro, usa **A* para gerar as vielas** e **quadtrees para organizar lotes** sobre terreno arbitrário, implementado em Unity. É a referência metodológica mais próxima do objetivo de geração procedural de favela.
- **MIT Senseable City Lab, "Favelas 4D"** (arXiv 2105.03235), análise de morfologia da Rocinha por laser scanning terrestre. E **"Unveiling the internal structures of informal settlements through AI-driven**

**automated segmentation of LiDAR data"**, npj Urban Sustainability, dezembro de 2025, sobre o Vidigal, com segmentação semântica 3D em sete classes: ground, vegetation, building, wire, rock, pole e movable objects. Esse esquema de classes é diretamente reaproveitável.

- **"Mapping Rio de Janeiro's favelas: general-purpose vs. satellite-specific neural networks"** (arXiv 2510.03725, 2025).

> **Prior art que você precisa conhecer com urgência.** Kim, Y.; Park, J.; Yoon, D.; Jung, N.; Jeon, M. **"G-MAD: A Game-Based Data Generation Framework for Multi-View RGB-T Aerial Object Detection"**, aceito no **ACM Multimedia 2026** (arXiv 2607.19942). Usa **ARMA 3** para gerar dados aéreos **RGB-T** multi-vista sincronizados, com **anotação automática de bounding box a partir de metadados geométricos do engine**, e libera código e o benchmark AMOD. É exatamente a metodologia do seu projeto de 2022, agora publicada numa conferência de primeira linha. E o ACM MM 2026 acontece **no Rio de Janeiro, de 10 a 14 de novembro de 2026**, no Windsor Barra, primeira vez na América Latina. Recomendação: citar obrigatoriamente, avaliar reusar o código em vez de reconstruir o pipeline, e considerar ir ao evento.

## 4. A restrição de hardware, que é hoje o seu maior gargalo

Verificado na máquina em 12/09/2026: **MacBook Air M1, 8 GB de memória unificada, GPU de 7 núcleos, 64 GB livres em disco**. Blender, Unity e Unreal não estão instalados. Não há GPU NVIDIA nem CUDA disponível.

Isso não é um detalhe de configuração, é o que elimina a maior parte das opções.

| Requisito | Sua máquina | Veredito |
|---|---|---|
| Unreal Engine 5.8, mínimo oficial em macOS: 16 GB RAM, recomendado 32 GB | 8 GB | **Abaixo do mínimo** |
| Nanite e Virtual Shadow Maps em macOS: "Apple Silicon M2+ (Beta support)" | M1 | **Não suportado** |
| Path Tracer da Unreal em macOS | Metal | **Não existe.** Engenheiro da Epic no fórum da Apple: *"Until Apple officially supports raytracing/path tracing, there's nothing we can do on our side for now."* |
| Ray tracing e path tracing do Unity HDRP: DirectX 12 apenas | Metal | **Não existe no Mac** |
| Blender Cycles, cena urbana densa | 8 GB unificados | Estoura. Há bug conhecido de consumo de memória no M1 com BVH2 (MetalRT só é padrão em M3+), e combinar CPU+GPU no Apple Silicon multiplica o uso. Fontes recomendam 24 GB ou mais |
| Isaac Sim / Omniverse Replicator: RTX com RT Cores, Linux ou Windows | Apple Silicon | **Impossível.** Nem A100 nem H100 servem, por falta de RT Cores |
| CARLA, Project AirSim, Cosys-AirSim | macOS | **Não rodam** |
| BlenderProc | macOS ARM | Roda nativo (baixa Blender 4.2.1 arm64), mas **sem GPU no macOS**, cai em CPU |
| Treino de 3D Gaussian Splatting (gsplat, SuGaR, 2DGS) | sem CUDA | Fora, com a exceção do **Brush**, que roda via WebGPU em Metal |

> **Conclusão de hardware.** O Mac serve para modelagem leve, organização de assets, scripts, análise e escrita. Não serve para renderizar o dataset nem para rodar engine com iluminação avançada. A saída não é comprar máquina, sobretudo porque 2026 é um péssimo momento: há crise de GDDR7 por demanda de datacenter de IA, e a RTX 5090 está entre R$ 20.000 e R$ 27.650 no Brasil, contra MSRP original de US$ 1.999. A saída é **alugar GPU por hora**.

| Opção de nuvem | Preço aproximado | Observação |
|---|---|---|
| Vast.ai, RTX 4090 | US$ 0,11 a 0,29/h | Mais barato, menos previsível |
| RunPod Community, RTX 4090 | US$ 0,34/h | Bom equilíbrio |
| RunPod, RTX A6000 48 GB | US$ 0,33/h | Mais VRAM pelo mesmo preço |
| AWS g6.xlarge (L4 24 GB) | US$ 0,80/h | A mais barata da AWS com RT cores |
| Render farm por frame (GarageFarm, RebusFarm) | US$ 0,004 a 0,072 por unidade | Modelo ruim para dezenas de milhares de imagens. Serve para lote final calibrado, não para tatear configuração |

A conta prática é tranquilizadora: 50 mil imagens em Cycles a 15 s por frame dão cerca de 208 horas de GPU, o que a US$ 0,40/h sai por volta de **US$ 83**. **Renderizar não é o gargalo de custo.** O gargalo é tempo humano de modelagem, limpeza de malha e delighting.

## 5. Comparação dos softwares

### 5.1 Quadro geral

| Critério | Blender 5.2 LTS | Unreal Engine 5.8 | Unity 6.5 / 6.6 |
|---|---|---|---|
| Custo | Grátis, GPL | Grátis para pesquisa e não comercial. Royalty de 5% só acima de US$ 1M em jogos | Personal grátis até US$ 200 mil de receita |
| Qualidade máxima de imagem | **A melhor**, Cycles é path tracer sem compromisso | Muito alta. Lumen, Nanite, VSM, MegaLights, Substrate, e Path Tracer para imagem final | Alta com HDRP, mas o HDRP entrou em **modo de manutenção em 23/02/2026**, sem novas features para sempre |
| Velocidade de render em lote | Lenta (segundos a minutos por frame) | **Muito rápida** (frações de segundo a segundos) | Muito rápida |
| Geometria densa e irregular | Sem equivalente a Nanite. Instancing e linked libraries ajudam, mas há teto | **Nanite** resolve. Comprime a ~14,4 bytes por triângulo; mesh de 1,5M tri ocupa 19,6 MB contra 149 MB tradicional | **Sem geometria virtualizada.** Budget de polígono e LOD manual continuam obrigatórios |
| Anotação automática para CV | Via **BlenderProc**, maduro: COCO e BOP prontos | Via EasySynth, UnrealCV, Cosys-AirSim. Bbox derivado de máscara semântica | Unity Perception, **oficialmente descontinuado** |
| Roda no seu Mac M1 8 GB | Parcialmente, com dor | **Não** (abaixo do mínimo, e Nanite exige M2+) | Sim para rasterização, **não** para ray/path tracing |
| Integração com Claude | blender-mcp (28,3k estrelas) e **MCP oficial da Blender Foundation** (v1.0.3, 11/09/2026) | **Plugin MCP oficial da Epic na 5.8**, com skills de PCG | **Plugin oficial da Unity para Claude Code (10/09/2026)**, 29 skills, Unity CLI |
| Assets de favela disponíveis | KitBash3D Favelas | **Os melhores**: Leartes (531 meshes), KitBash3D, Megapack Rocinha | Leartes com HDRP, Brazilian Favela Modular Pack |

### 5.2 Blender: onde ganha e onde trava

Versão atual: **5.2 LTS**, lançada em 14/07/2026 (a 5.0 saiu em 18/11/2025). O Cycles ganhou texture cache na linha 5.x, o que ajuda justamente em cena pesada de textura como malha fotogramétrica.

Ganha em: qualidade final de imagem sem concessão, custo zero, controle total por Python, e o ecossistema de dado sintético mais maduro em código aberto (BlenderProc). Para gerar figuras de artigo com luz fisicamente correta, é a melhor ferramenta da lista.

Trava em: ausência de geometria virtualizada, o que impõe teto real em cena urbana densa; velocidade de render em lote; e, na sua máquina, memória. Há bug documentado de consumo de memória no M1 com BVH2, sendo que MetalRT só virou padrão a partir do M3.

> **Gotcha crítico do BlenderProc.** O instalador está preso a **Blender 4.2.1 LTS** por código explícito (`major_version = "4.2"`, `minor_version = "1"` em `InstallUtility.py`). Não suporta 4.3, 4.5 nem 5.x. Ou seja, escolher BlenderProc significa abrir mão dos ganhos do Cycles 5.x. Versão 2.8.0, último release em outubro de 2024, commits até janeiro de 2026: vivo mas lento. E em macOS ele cai em CPU, sem GPU.

### 5.3 Unreal Engine: a escolha tecnicamente mais forte, e a que você não consegue rodar

Versão atual: **5.8, lançada em 17/06/2026**, e é declaradamente o último lançamento maior da linha UE5. A Epic já migrou o esforço para a UE6, com Early Access previsto para o fim de 2027. Para um projeto começando agora, mirar 5.7 (mais estável) ou 5.8 (mais recursos) é razoável, sabendo que migrar para UE6 em 2028 não será trivial.

É o que o paper de referência da seção 2 usou (na versão 5.1), é onde estão os melhores assets de favela, tem a renderização em lote mais rápida, e tem Nanite, que é a única resposta real ao problema de milhares de volumes irregulares interpenetrados. O Lumen em modo *Software Ray Tracing* é particularmente adequado aqui porque o custo independe do número de instâncias e da sobreposição entre elas, ao contrário do modo por hardware, cuja documentação cita explicitamente *"kit-bashed overlapping layers"* como caso problemático. Uma favela é, por definição, camadas sobrepostas.

Pontos de atenção levantados na pesquisa: tessellation e displacement do Nanite continuam experimentais na 5.8 e há regressão reportada e confirmada por múltiplos usuários (funciona na 5.7.4, quebra na 5.8 e 5.8.1), o que importa para tijolo aparente com relevo real. Lightmaps assados com Nanite são tecnicamente frágeis e não recomendados. E Virtual Shadow Maps têm custo dominado por invalidação de cache, que dispara com qualquer coisa animada: roupa no varal, lona ao vento, folhagem.

### 5.4 Unity: perdeu terreno, e para o seu caso específico perdeu mais ainda

Versão atual: 6.6 (31/08/2026), com 6.3 como LTS. O Unity 7 foi anunciado para GA no primeiro trimestre de 2027.

Três fatos, verificados, que desaconselham o Unity neste projeto:

1. **O HDRP entrou em modo de manutenção em 23/02/2026.** Comunicado oficial: manter o HDRP "focusing only on expanding platform reach with Nintendo Switch 2 support and stability". Nenhuma feature nova planejada. E a tabela oficial de comparação do Unity 6.3 mostra que o URP, que é o futuro, **não tem** ray tracing, path tracing, physically based sky, volumetric clouds, SSGI, SSR, subsurface scattering, physical light units, nem physical camera completa. Para fotorrealismo hoje o HDRP é a única opção, e é tecnologia congelada.
2. **O Unity Perception está oficialmente descontinuado.** O README diz literalmente: *"This project has been discontinued and is no longer supported by Unity."* Último commit de código em 28/11/2022, última release `1.0.0-preview.1`, nunca saiu de preview, zero pull requests mergeados desde 2022. A linha de produto "Unity Computer Vision" sumiu do site (as URLs redirecionam para `unity.com/products`), e o Unity Simulation não existe mais. Existe um fork comunitário vivo que resolve o Unity 6 (`tobeornottobia/com.unity.perception`, branches `upgrade-unity6.1` e `upgrade-unity6.5`, commit em 01/09/2026), com diff pequeno (14 arquivos, +57/-39), mas é um autor, zero estrelas, sem CI. A toolchain Python associada (`pysolotools`) teve último upload no PyPI em 2022 e pina numpy 1.23, que não instala em Python moderno.
3. **Ray tracing e path tracing do HDRP são DirectX 12 apenas**, ou seja, Windows. No Mac você fica sem justamente o recurso que daria qualidade de referência ao dataset.

Contexto de mercado, para calibrar: a pesquisa State of the Game Industry da GDC 2026, com mais de 2.300 profissionais, registrou **pela primeira vez a Unreal passando a Unity como engine primária, 42% contra 30%**. No fórum oficial da Unity, a reação de profissionais ao anúncio de pipelines foi dura, incluindo *"HDRP is the only thing that gets on-par with AAA graphics... If HDRP gets dropped, I am leaving Unity"* e *"URP is a decade behind HDRP and is unlikely to catch up anytime soon"*.

> Há um caso interessante e diretamente aplicável, apesar de tudo: Dergachov e Aizatskyi (WDA 2026, CEUR-WS Vol-4155) usaram Unity + Perception + HDRP com câmera em trajetórias Bezier simulando voo de drone, detecção de carro, baseline VisDrone e YOLOv8n. Resultado que vale memorizar: **400 imagens reais (7,6% do VisDrone) mais 6.289 sintéticas atingem 93% do mAP do treino totalmente real**. E melhorar a qualidade do sintético entre iterações reduziu em 8 vezes o dado real necessário. Também acharam saturação em 700 a 1.000 imagens por localização: mais volume sem mais diversidade não adiciona nada.

### 5.5 Alternativas fora do trio

| Stack | Estado em 2026 | Veredito para você |
|---|---|---|
| **ARMA 3** (o que você já usa) | Vivo, e agora com precedente publicado no ACM MM 2026 (G-MAD) | **Não descarte.** Já funciona, você tem o código, e a justificativa dos autores do G-MAD é exatamente a sua: assets prontos eliminam custo substancial de criação. O que falta é a favela, que o ARMA não tem |
| Isaac Sim 6.x + Replicator (NVIDIA) | Muito ativo, open source Apache 2.0, gratuito desde maio de 2026. Isaac Sim 6.0 traz **NuRec**, com Gaussian Splatting integrado e **interação de luz entre splat e mesh** | Maior teto de qualidade de anotação (bbox 3D nativo, oclusão, KittiWriter). Mas exige RTX com RT Cores, Linux ou Windows, e pipeline USD. Só vale se o LiDAR vier e você quiser casar splat real com geometria autoral |
| CARLA 0.9.16 / branch ue5-dev | Ativo (14,4k estrelas). Linha UE 5.5 parou na 0.10.0 de dez/2024; trabalho real em nightlies sem release tagueada | O repositório `LTTM/Syndrone` publica **o gerador**, não só o dataset: CARLA modificado, 28 classes, alturas 20/50/80 m e pitch 30/60/90 já parametrizados. É o seu caso de uso com código pronto, mas exige Linux e não tem favela |
| Project AirSim (IAMAI, ex-Microsoft) | **Vivo sob licença MIT**, v1.0.1 em 07/09/2026, iniciativa apoiada pela DARPA. Suporta UE 5.2 ou 5.7 | O `microsoft/AirSim` original está morto desde 2022 e o **Colosseum foi arquivado**. Se o plano era usar o fork "oficial", ele acabou. Cosys-AirSim (release para UE 5.8) é o fork vivo mais direto |
| Infinigen (Princeton) | Muito ativo, 7,2k estrelas, v2.0.0a2 em 27/08/2026 | **Não tem gerador urbano.** Só natureza e interiores. E o custo publicado no CVPR 2023 é de **3,5 horas por par de imagens 1080p** a 10.000 amostras por pixel. Não copie a configuração de render deles |
| Kubric (Google) | Manutenção mínima, sem releases formais | Escopo errado (objetos caindo numa caixa). Ignore |

> **Achado que sustenta a tese do artigo.** Os três principais datasets aéreos sintéticos (SkyScenes, SynDrone, UEMM-Air) são construídos sobre mapas CARLA ou Unreal de cidade genérica de padrão norte-atlântico. O Infinigen gera natureza e interiores, não cidade. Ou seja, **a própria infraestrutura de dados sintéticos codifica uma morfologia urbana específica**, e nenhuma delas é a do Sul Global. Além disso, nenhuma das buscas encontrou trabalho publicado gerando imagem aérea sintética de favela ou assentamento informal com game engine. A literatura de mapeamento de favela é quase toda satélite com dado real. Isso é lacuna verificável e é um argumento direto para "Machine Vision from the Global South", não apenas retórica.

## 6. Sol, sombra e luz: a parte que você intuiu certo

A Ablação A da seção 2.2 mostrou que remover variação de iluminação e de ângulo custa 10 pontos de mAP no geral e 20 em objetos truncados. Vale, portanto, tratar isso com rigor e não no olho.

### 6.1 Parâmetros corretos para o Rio de Janeiro

| Parâmetro | Valor | Cuidado |
|---|---|---|
| Latitude | **-22,9068** | Negativo para o hemisfério Sul |
| Longitude | **-43,1729** | Negativo para oeste de Greenwich |
| Fuso horário | **-3** (UTC-3) | |
| Horário de verão | **Desligado** para qualquer data a partir de 2019 | O Brasil fez a última mudança de relógio em 16/02/2019 e aboliu o horário de verão por decreto em abril de 2019. Para datas anteriores, a janela era de outubro a fevereiro e precisa ser configurada à mão, porque os defaults dos plugins seguem convenção do hemisfério norte |
| Orientação do modelo (North Offset) | Calibrar | **É a armadilha maior, e não é o fuso.** Se a geometria estiver posicionada arbitrariamente na cena, as sombras caem nas paredes erradas mesmo com azimute correto. Num morro, onde a orientação da encosta determina quais casas pegam sol da manhã e quais ficam na sombra o dia inteiro, isso é a diferença entre estudo válido e erro sistemático |

### 6.2 Ferramentas e precisão declarada

- **Blender, addon Sun Position** com textura de céu Nishita: coordenadas, data e hora. Gratuito e embutido.
- **Unreal, Sun Position Calculator** (plugin em Misc): retorna `elevation`, `corrected_elevation` (corrigida para refração atmosférica), `azimuth`, `sunrise_time`, `sunset_time` e `solar_noon`. A Epic **não publica número de precisão** nem declara o algoritmo. Exposto ao Python como `unreal.SunPositionFunctionLibrary`, o que permite varrer horas e datas por script.
- **Ultra Dynamic Sky** (Fab, a partir de US$ 49,99, UE 5.5 a 5.8): é o único do ecossistema com **erro publicado**, de cerca de **0,03 grau** para sol e lua, com fórmulas baseadas no *Astronomical Algorithms* de Jean Meeus. Foi o plugin usado no paper de referência da seção 2.
- **Referência de comparação:** o NREL Solar Position Algorithm (Reda e Andreas) declara incerteza de 0,0003 grau.

### 6.3 Como validar, em três camadas

1. **Efeméride independente.** Comparar azimute, elevação corrigida, nascer, pôr e meio-dia solar com NOAA Solar Calculator, suncalc.org ou `pvlib` / `pysolar` em Python, para a mesma coordenada, data e hora. Diferença acima de 0,1 grau denuncia erro de fuso, DST ou data.
2. **Sombra medida em campo.** Fotografar um objeto vertical conhecido com timestamp, medir comprimento e direção da sombra real e reproduzir na cena. **É o único teste que pega erro de orientação do modelo**, que a camada 1 não pega.
3. **Fotometria.** Existe precedente acadêmico ("Validating Game Engines as a Quantitative Daylighting Simulation Tool") que mediu iluminância em pontos selecionados na Unreal contra ferramenta validada e luxímetro físico, com erro médio próximo ao da simulação de referência.

### 6.4 Exposição fisicamente baseada, para casar com foto real

Fórmula: **EV100 = log2(abertura² / tempo de obturador × 100 / ISO)**. Dia ensolarado a f/16, ISO 100 e 1/125 s dá EV100 = 15.

Procedimento: desligar auto-exposure (senão ela injeta variável espúria entre frames do dataset), colocar a luz direcional em **100.000 lux** para sol de meio-dia (o CesiumSunSky usa ~111.000), e derivar o EV alvo do EXIF da foto de referência. Na Unreal é preciso ligar "Extend default luminance range" nas configurações de renderização, senão os valores físicos não se comportam como esperado.

Dois detalhes que denunciam uma imagem sintética na hora quando errados:

- **Ângulo angular do sol.** O sol real tem cerca de 0,545 grau de diâmetro angular, e é isso que controla a dureza da penumbra. Penumbra errada é o sinal mais diagnóstico ao comparar com foto real. No Rio, a luz de meio-dia é dura e quase vertical, e a de fim de tarde é rasante, que é justamente a condição mais bonita e a mais difícil de renderizar sem artefato (aliasing projetivo em ângulo raso é problema conhecido de Virtual Shadow Maps).
- **Nuvem não é previsão.** Volumetric clouds são fisicamente plausíveis, mas nenhum engine ingere METAR ou imagem de satélite. Para reproduzir uma data e hora específicas, a cobertura precisa ser autorada a partir da foto de referência.

## 7. Texturas, materiais e o trabalho invisível que vai dominar o cronograma

### 7.1 O problema do delighting

> A textura da sua fotogrametria de 2022 tem **iluminação e sombras assadas no albedo**. Cada sombra projetada, cada oclusão de ambiente, cada realce especular daquele dia e daquela hora está pintado na imagem. Mudar a posição do sol não apaga a sombra que já está na laje. Para variar o sol de forma fisicamente coerente, é preciso recuperar o albedo puro. **Este é o maior trabalho invisível do projeto e precisa ser planejado explicitamente.**

Os próprios autores do paper de referência declaram isso como limitação: *"while photogrammetry ensures geometric consistency, the interaction between simulated lighting and reconstructed textures inevitably approximates real-world optics. This discrepancy may lead to missed detections under extreme lighting or complex material reflectance conditions."*

Ferramentas disponíveis:

- **Agisoft De-Lighter**: gratuito, standalone, remove sombras projetadas e oclusão de ambiente das texturas fotogramétricas. O caminho mais direto.
- **Unity DeLightingTool** (github.com/Unity-Technologies/DeLightingTool): mais controle, mas exige diffuse lit, normais em object space, bent normals e AO.
- Abordagens recentes de decomposição intrínseca com difusão (por exemplo DiffusionLight), ainda experimentais.

Mitigação parcial e barata: se o LiDAR do IPP vier, a geometria vem sem textura assada, e você texturiza com materiais PBR autorais amostrados das distribuições reais observadas na ortofoto de 15 cm. Isso resolve o delighting por construção, ao custo de perder o detalhe fotográfico da fachada.

### 7.2 Fontes de textura

- **Megascans**: grátis para todos apenas até 31/12/2024. Quem reclamou o acervo naquela janela mantém uso perpétuo. Desde 2025 a Epic voltou a cobrar por asset (a partir de US$ 0,99). **Porém**, o acesso ilimitado continua válido para quem tem licença da Unreal Engine, com a contrapartida de que esses assets são "UE-Only Content" e não podem ser renderizados fora da engine.

- **Poly Haven** e **ambientCG**: CC0, sem restrição, ideais para concreto, reboco, telha de fibrocimento, ferrugem e metal corrugado.
- **Substance 3D Sampler**: tem Text-to-Texture, Text-to-Pattern e Image-to-Texture em beta, gerando tileable com Base Color, Metallic, Roughness e Normal. A jogada realista para favela é fotografar fachadas reais e passar por ali. **ArmorLab** é a alternativa gratuita e offline.
- Não encontrei nenhum pack de **tipografia e sinalização comercial brasileira** (letreiro de bar, placa de comércio, grafismo carioca), que é um dos elementos mais identitários. Isso provavelmente precisa ser feito à mão a partir de fotos.

### 7.3 A lição de quem já construiu favela fotorrealista

Jean Pierre Vargas construiu um diorama de favela em UE5 com Lumen e Nanite, em seis meses de trabalho parcial, com Maya, Substance 3D Painter, Quixel Mixer, Marvelous Designer e Marmoset. O achado metodológico mais importante do relato dele: **ele rejeitou explicitamente a abordagem modular**. Nas palavras dele, "the more I worked on the project and analyzed the reference images, the more I realized that this approach wouldn't be very suitable", porque as unidades superiores avançam sobre as inferiores de forma irregular. Modelou cada apartamento como mesh separado.

A estrutura de quatro andares dele progride de concreto acabado embaixo para tijolo aparente em cima, mais um puxadinho de madeira, refletindo expansão geracional. O shader usa canais de UV secundários com máscaras RGB no térreo para variar cor de pintura e concreto exposto, e vertex paint nos andares superiores misturando tijolo, concreto rugoso e mofo por height lerp, com uma máscara geral de sujeira. Iluminação: uma única luz direcional com skylight e sky atmosphere, Lumen dispensando qualquer bake.

Outros dois projetos de referência: Gian Franco Caserta fez favela em UE 5.3 com Kitbash e Quixel usando **PCG para distribuição aleatória de assets**, e "SCARCITY", de Yuri Vieira de Aguiar, é um ambiente real-time em UE4 inspirado em Pavão-Pavãozinho e Cantagalo.

## 8. Claude no processo: o que funciona e o que é demo

### 8.1 O dado mais honesto disponível

O benchmark **3DCodeBench** (arXiv 2606.01057, junho de 2026) mede modelos escrevendo código 3D. Dois resultados importam:

| Modelo | Executabilidade | Elo humano de qualidade |
|---|---|---|
| GPT-5.5 | 0,906 | 1.163 |
| Gemini 3.1 Pro | 0,725 | 1.147 |
| **Claude Opus 4.7** | **0,910 (o maior)** | 1.006 (o menor dos frontier) |

Leitura: Claude é o mais confiável em produzir código que roda sem quebrar, e o mais fraco em qualidade geométrica percebida. Isso define exatamente o papel certo dele: **engenheiro de pipeline e autor de código procedural, não artista**.

E o achado decisivo do mesmo benchmark: dar feedback de erro em múltiplos turnos leva a executabilidade de 0,702 para 0,974, **mas a qualidade condicional da forma quase não melhora**. O loop de erro conserta crash. Não conserta geometria desconectada, desalinhada ou feia.

### 8.2 Onde os MCPs estão em setembro de 2026

| Ferramenta | Estado | Nota |
|---|---|---|
| **MCP oficial da Blender Foundation** | v1.0.0 em 27/04/2026, v1.0.3 em 11/09/2026 | Foco declarado em interface de linguagem natural para a API Python, mais copiloto de API do que modelador. Já tem captura de viewport |
| **ahujasid/blender-mcp** | 28.324 estrelas, push em 07/09/2026 | O mais popular. Poly Haven, Sketchfab, Poly Pizza, Hyper3D Rodin, Hunyuan3D. **Ver alerta de segurança abaixo** |
| **MCP oficial da Epic (UE 5.8)** | Experimental, servidor no editor em `127.0.0.1:8000/mcp` | Toolsets de cena, mesh, material, PCG e captura de viewport. Sem autenticação, loopback apenas. A Epic pretende torná-lo "an integral part of UE6" |
| **Plugin oficial da Unity para Claude Code** | Publicado em **10/09/2026**, 29 skills | Via Unity CLI (`brew install --cask unity-cli`). Grátis, roda offline, não consome Unity Points. Mas as skills de rendering são URP, não HDRP |
| CoplayDev/unity-mcp | 14.148 estrelas, v10.2.0 em 01/09/2026 | 47 tools. O ecossistema Unity é hoje o mais maduro dos três |
| chongdashu/unreal-mcp | 2.077 estrelas, **parado desde 22/04/2025**, sem licença | **Não use.** É o mais citado em blogs e o mais desatualizado |

> **Alerta de segurança sobre o blender-mcp.** Em 09 e 10/08/2026 a conta GitHub do mantenedor foi comprometida e os repositórios foram transferidos para outra organização (depois recuperados). Há CVEs abertos contra o projeto: CVE-2026-10661 (injection), CVE-2026-10662 (SSRF), CVE-2026-10688 (code injection), além de leitura arbitrária de arquivo. O mantenedor adicionou um "safe mode" em 02/09/2026 e há Dockerfile desde 05/09/2026. Como a ferramenta executa Python arbitrário na máquina e baixa assets de URLs, rode em container ou em safe mode.

### 8.3 O que a comunidade relata, sem filtro

O relato mais próximo do seu caso de uso, sobre alimentar desenhos 2D para gerar arquitetura 3D via MCP: *"No understanding on how windows attach to walls, misaligned bearing walls between floors, gaps all around the places, chimney not continuous, windows don't line up and are in wrong places, house parts swapped. And still struggling when I explain those things... But I just had to keep prompting to fix this, that, redo this and that and then it breaks other things until I fed up."*

No BlenderArtists, numa thread dedicada ao tema em setembro de 2025, praticamente nenhum artista profissional tinha testado, e um deles resumiu: *"I didn't see any practical examples that looks good or useful."* As 28 mil estrelas vêm do público de desenvolvimento e IA, não do público de 3D.

Um teste de custo documentado: reproduzir o tutorial do donut do Blender Guru via MCP consumiu 60% dos tokens de sessão de um plano de US$ 200 por mês, levou duas horas, e o render final tinha objetos atravessando outros e ângulo de câmera errado. Um donut.

Limites estruturais bem identificados: o modelo só conhece o que as tools expõem; cada ação é uma ida e volta (um passe de limpeza em 40 objetos com 10 operações vira 400 chamadas); operações pesadas estouram timeout porque o protocolo não tem vocabulário para trabalho assíncrono; e nomes de sockets de Geometry Nodes mudam entre versões.

### 8.4 O padrão que de fato funciona

Não é MCP conversacional, é **script headless com loop de validação**. O caminho: `blender --background --python script.py`. É a API completa, sem GUI, sem socket, sem ida e volta por operação, determinístico, versionável em git, e paralelizável.

Sobre a validação visual, a resposta é sim, e há workflows documentados. O padrão consolidado:

1. **Índice da API antes de agir.** Fornecer um arquivo de referência da API da sua versão exata, em vez de confiar na memória do modelo. No 3DCodeBench, a maior causa de falha de execução foi incompatibilidade de API com o Blender 5.0, porque o modelo escreve código da versão que estava no treino.
2. **Todo script escreve um render de preview**, e o agente lê a imagem. Três ângulos fixos por iteração: topo (layout e sobreposição), altura dos olhos (estética) e o ângulo de captura real.
3. **Anotações no render**: grade de coordenadas e rótulos de objeto. O agente raciocina muito melhor sobre "o objeto X está em (3, 0, 12)" via JSON do que sobre pixels.
4. **Sidecar JSON compacto** junto do PNG. Nunca jogar blob base64 grande no contexto.
5. **Comparação contra referência** (moodboard ou foto real), em vez de crítica aberta.
6. **Validação numérica junto da visual**: contagem de polígonos, bounding boxes, testes de interpenetração, normais, escala em metros, densidade de texel. O agente é ruim de olho e bom com números.

Confirmação acadêmica de que essa direção é a certa: **LL3M** (arXiv 2508.08228), sistema multiagente com planner, retrieval, coding, critic e verification, escrevendo `bpy` com RAG da API do Blender, supera a baseline BlenderMCP. E **"Thinking in Blender"** (arXiv 2606.02580, junho de 2026) reconstrói cena 3D a partir de uma única imagem como programa Blender editável, refinando em estágios (geometria, materiais, composição, iluminação), e conclui que *"staged reconstruction substantially improves reconstruction fidelity, highlighting the importance of task decomposition."* Isso é literalmente "foto de prédio de favela para cena Blender editável", mas é paper recente, sem produto.

### 8.5 Geração de assets por IA: onde entra e onde não entra

O bake-off mais útil encontrado construiu o mesmo prédio por seis métodos e concluiu, textualmente: *"CC0 kits win for the repeated city mass; Blender headless for bespoke shapes; image-to-3D only for one-off props*

*(messy topology, as predicted)."* E o insight estratégico: *"Architect the city from real footprint extrusion, NOT AI-generated buildings."*

Traduzindo: text/image-to-3D (Meshy 6, Tripo, Hyper3D Rodin, Hunyuan3D, TRELLIS) **não serve para os prédios**. Serve razoavelmente para props avulsos (caixa d'água, ar-condicionado, antena parabólica, botijão, cadeira de plástico), e mesmo assim com retopologia, reescala (chegam com fator ~10x errado) e correção de eixo (Y-up contra Z-up).

Sobre world models generativos: **World Labs Marble** é o único que exporta artefato editável (splat .ply, collider mesh e GLB texturizado de ~600k triângulos), com planos de US$ 20 a 95 por mês, mas a própria documentação declara "reconstruction artifacts, with uneven geometry, holes, and blobby or 'floater' surfaces". Serve como blockout e referência de layout, não como quarteirão final. **Genie 3, Odyssey e Decart não exportam geometria**: são render neural ou vídeo, inúteis para este pipeline.

## 9. Gaussian Splatting: por que não resolve o seu problema

Vale tratar explicitamente, porque é a tecnologia da moda e parece a resposta óbvia para "pegar o real e colocar no computador".

Existe pesquisa madura para cena urbana grande a partir de imagem aérea: Scaffold-GS (arXiv 2312.00109), VastGaussian (2402.17427), Octree-GS (2403.17898), CityGaussian (2404.01133), Hierarchical 3DGS (2406.12080), CityGaussianV2 (2411.00771) e HUG (2504.16606), este último especificamente para urbano aéreo em blocos.

> **O bloqueio é iluminação.** Um splat codifica a aparência exatamente como foi fotografada, com luz assada nos harmônicos esféricos de cada gaussiana. Não existe material PBR nem shader interagindo com luz. Mover o sol não apaga a sombra pintada na laje. Como a variação de sol vale 10 a 20 pontos de mAP (Ablação A), **3DGS não serve como motor de variação de iluminação**, que é exatamente o que você precisa.

Há pesquisa de relighting (GS-IR arXiv 2311.16473, Relightable 3D Gaussians 2311.16043, GaRe 2507.20512, e ROSGS 2509.11275, o mais relevante para cena externa). Mas a própria literatura declara que os métodos atuais "carecem de flexibilidade para mudar o horário de captura ou ajustar condições climáticas, e tais mudanças frequentemente levam a imprecisões em iluminação e predição de sombra". E há um limite que ninguém escreveu porque ninguém testou: **nenhum desses métodos foi validado em favela vertical densa**, que é o pior caso possível para rendering inverso, com auto-oclusão extrema, milhares de superfícies pequenas em ângulos arbitrários e sombra projetada dominante.

Onde 3DGS serve neste projeto: validação visual da reconstrução, geração de vistas próximas às originais, e figura de qualidade no artigo. No seu Mac, a única ferramenta séria que roda é o **Brush** (WebGPU sobre Metal, sem CUDA).

Uma ideia aproveitável de Niżeniec e Iwanowski (arXiv 2512.13411): eles usam 3DGS para o ambiente e aplicam **two-pass rendering combinando o splat com shadow maps derivados de proxy meshes**, recuperando sombras plausíveis sobre um splat sem material, com máscaras de segmentação automáticas em formato YOLO. É um meio-termo elegante se você quiser o realismo do splat sem abrir mão de sombra controlável.

## 10. Recomendação

### 10.1 O caminho recomendado

> **Fotogrametria ou LiDAR real do Jacarezinho como base geométrica, detalhe paramétrico por cima, renderização com randomização estruturada de sol, clima e ângulo, e anotação automática por máscara de instância.** Essa é a arquitetura validada por Liao et al. (2026) com ganho medido de 22,5 pontos sobre assets genéricos e 10 a 20 pontos sobre iluminação estática.

Sobre o software, a resposta honesta é que **ela depende de uma informação que você ainda não tem**: se o IPP libera o LiDAR de 2024. Mas a escolha de engine é menos crítica do que parece, porque o fator dominante é a geometria, não o renderizador. Dito isso:

| Cenário | Stack recomendado | Razão |
|---|---|---|
| **Prioridade em publicar rápido e reaproveitar o que existe** | ARMA 3 + código do G-MAD | Você já domina, já tem scripts, e agora há precedente publicado no ACM MM 2026 com código aberto. Falta a favela, que é o que você acrescentaria de original |
| **Prioridade em fidelidade e o LiDAR vier** | Blender 4.2.1 + BlenderProc, renderizando em GPU alugada | Custo zero de licença, anotação COCO pronta, melhor qualidade de luz, e roda no seu fluxo sem depender de máquina Windows para autorar |
| **Prioridade em volume de imagens e realismo de engine** | Unreal 5.7 + EasySynth ou Cosys-AirSim, em máquina alugada | Duas ordens de grandeza mais rápido em lote, melhores assets de favela, e é o que o paper de referência usou |
| Unity | **Não recomendado neste projeto** | HDRP congelado, Perception descontinuado, ray tracing indisponível no Mac. O plugin oficial para Claude Code é excelente, mas não compensa o resto |

### 10.2 Roteiro sugerido, em ordem

1. **Hoje.** Escrever para `cartografia.ipp@prefeitura.rio` pedindo a nuvem LiDAR e o MDS do voo de 2024 para o recorte do Jacarezinho, como pesquisador de mestrado da FGV EBAPE. É a ação de maior alavancagem e tem latência de dias ou semanas, então precisa começar antes de tudo.
2. **Esta semana.** Ler o G-MAD (arXiv 2607.19942) e baixar o código e o benchmark AMOD. Decidir se reusa ou reconstrói. Ler o Liao et al. (DOI 10.3390/rs18020361) inteiro, é o template do método.
3. **Enquanto espera o IPP.** Baixar recortes da ortofoto 2024 de 15 cm do ImageServer público para o Jacarezinho. Isso já dá textura, footprints e, principalmente, as **distribuições reais** de cor de telhado, material e densidade construtiva, que são o insumo da randomização estruturada.
4. **Prova de conceito mínima.** Uma quadra, não a favela inteira. Renderizar 500 imagens variando só sol (três horários) e altitude (três alturas). Treinar YOLO. Medir. Isso valida o pipeline inteiro por menos de US$ 5 de GPU alugada e responde se a abordagem funciona antes de investir meses.
5. **Só então** decidir escalar, e para qual engine.

### 10.3 O que não fazer

- Não comprar kit de favela para servir de base geométrica. É o braço perdedor medido do experimento (22,5 pontos de mAP). Compre para props e detalhe, se comprar.
- Não usar Google Photorealistic 3D Tiles. Está contratualmente vedado para detecção de objetos.
- Não apostar em Gaussian Splatting como motor de variação de iluminação.
- Não tentar rodar Unreal 5.8 nesse MacBook Air. Está abaixo do mínimo oficial e o Nanite exige M2 ou superior.
- Não comprar GPU em 2026 se puder evitar. Alugue.
- Não copiar a configuração de render do Infinigen (10.000 amostras por pixel, 3,5 horas por par de imagens). É escolha de paper, não de produção.
- Não usar `chongdashu/unreal-mcp` (parado desde abril de 2025, sem licença) nem contar com Genie 3, Odyssey ou Decart.
- Não prometer, em proposta ou artigo, que dado sintético substitui dado real. A literatura é consistente: ele multiplica o valor de pouco dado real.

### 10.4 Onde o Claude entra de verdade

Escrevendo e depurando o gerador paramétrico em `bpy` ou o grafo PCG, orquestrando o pipeline (baixar recorte, converter formato, importar, instanciar, variar, renderizar, converter anotação), escrevendo o conversor de máscara de instância para YOLO, e rodando o loop de validação render, ler imagem, comparar com

referência, corrigir. Expectativa realista: economiza entre 40% e 60% do tempo de pipeline e praticamente **nada** do tempo de direção de arte e julgamento visual.

## 11. Limitações deste levantamento

- Não confirmei disponibilidade real, custo nem termos de licença da nuvem LiDAR de 2024 do IPP. O servidor público expõe apenas os mosaicos de imagem, não a nuvem de pontos.
- Não verifiquei a cobertura e a qualidade do OpenStreetMap especificamente para o Jacarezinho.
- Os preços de assets na Fab, ArtStation e CGTrader vêm em parte de resultados de busca, porque essas lojas bloqueiam acesso automatizado. Confirme antes de comprar.
- A leitura das cláusulas de IA da EULA da Fab e da Unreal Engine é interpretação literal de contrato de adesão, **não é parecer jurídico**. Para publicar dataset aberto, consulte o setor competente da instituição.
- Estimativas de tempo de render por frame para Cycles, Isaac Sim e Unreal são ordens de grandeza, não medições. O número do Infinigen (3,5 h por par) é publicado.
- O Reddit estava inacessível durante a pesquisa. As citações de comunidade vêm de fóruns oficiais (Epic Developer Community, discussions.unity.com, BlenderArtists), Hacker News e blogs técnicos.
- Há uma inconsistência interna no paper de Liao et al.: a seção de resultados cita ganho médio de 22,5 pontos da reconstrução de alta fidelidade, e a discussão cita 23,3. Usei o valor da seção de resultados.
- Não encontrei nenhum trabalho publicado gerando imagem aérea sintética de favela com game engine. Isso é afirmação sobre ausência de evidência, baseada em buscas convergentes, e não prova definitiva de inexistência.

## 12. Referências principais

### Evidência que sustenta a recomendação


- Liao, F.; Liu, Y.; Xu, W.; Wang, X.; Liu, G.; Yang, K.; Li, J. (2026). Bridging the Sim2Real Gap in UAV Remote Sensing: A High-Fidelity Synthetic Data Framework for Vehicle Detection. *Remote Sensing*, 18(2), 361. https://doi.org/10.3390/rs18020361
- Kim, Y.; Park, J.; Yoon, D.; Jung, N.; Jeon, M. (2026). G-MAD: A Game-Based Data Generation Framework for Multi-View RGB-T Aerial Object Detection. ACM Multimedia 2026. https://arxiv.org/abs/2607.19942
- Berwaldt, N. L. P.; Bettker, R. V.; Pozzer, C. T. (2020). Procedural Generation of Favela Layouts on Arbitrary Terrains. SBGames 2020, pp. 136-144. https://doi.org/10.1109/SBGames51465.2020.00027
- Prakash, A. et al. (2019). Structured Domain Randomization. ICRA. https://arxiv.org/abs/1810.10093
- Tobin, J. et al. (2017). Domain Randomization for Transferring Deep Neural Networks. IROS. https://arxiv.org/abs/1703.06907
- Tremblay, J. et al. (2018). Training Deep Networks with Synthetic Data. CVPRW. https://arxiv.org/abs/1804.06516
- Zakharov, S. et al. (2022). Photo-realistic Neural Domain Randomization. ECCV. https://arxiv.org/abs/2210.12682
- Krump, M.; Stütz, P. (2023). Sensors, 23(7), 3769. https://doi.org/10.3390/s23073769
- Burdorf, S.; Plum, K.; Hasenklever, D. (2022). Reducing the Amount of Real World Data for Object Detector Training with Synthetic Data. https://arxiv.org/abs/2202.00632
- Rizzoli, G. et al. (2023). SynDrone. ICCVW. https://arxiv.org/abs/2308.10491 | Código: https://github.com/LTTM/Syndrone
- Khose, S. et al. (2024). SkyScenes. ECCV. https://arxiv.org/abs/2312.06719
- Liu, F. et al. (2024). UEMM-Air. https://arxiv.org/abs/2406.06230
- Kiefer, B.; Ott, D.; Zell, A. (2022). Leveraging Synthetic Data in Object Detection on UAVs. ICPR. https://arxiv.org/abs/2112.12252
- Dergachov, K.; Aizatskyi, O. (2026). Synthetic data for deep learning in object detection tasks. WDA'26, CEUR-WS Vol-4155. https://ceur-ws.org/Vol-4155/paper03.pdf
- Barisic, A. et al. (2022). Sim2Air. IEEE RA-L. https://arxiv.org/abs/2110.05145


### Favela, morfologia e dados do Rio


- MIT Senseable City Lab (2021). Favelas 4D. https://arxiv.org/abs/2105.03235
- Liu et al. (2025). Unveiling the internal structures of informal settlements through AI-driven automated segmentation of LiDAR data. *npj Urban Sustainability*. https://www.nature.com/articles/s42949-025-00295-9
- Mapping Rio de Janeiro's favelas (2025). https://arxiv.org/pdf/2510.03725

- IPP, ortofoto verdadeira 2024, 15 cm: https://pgeo3.rio.rj.gov.br/arcgis/rest/services/Imagens/Mosaico_2024/ImageServer
- Mapoteca IPP e solicitação de acervo: https://ipp.prefeitura.rio/home/mapoteca/ | cartografia.ipp@prefeitura.rio
- DATA.RIO, camada Edificações: https://www.data.rio/datasets/124f1607b57d4292af5c0dfc2ff8572c_0/about

## Ferramentas, documentação e licenças

- BlenderProc: https://github.com/DLR-RM/BlenderProc | Infinigen: https://github.com/princeton-vl/infinigen
- MCP oficial da Blender Foundation: https://www.blender.org/lab/mcp-server/ | blender-mcp: https://github.com/ahujasid/blender-mcp
- Unreal MCP oficial: https://dev.epicgames.com/documentation/unreal-engine/unreal-mcp-in-unreal-editor
- UE 5.8 release notes: https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-5-8-release-notes
- Sun Position Calculator (UE): https://dev.epicgames.com/documentation/unreal-engine/geographically-accurate-sun-positioning-tool-in-unreal-engine
- Plugin oficial da Unity para Claude Code: https://unity.com/blog/unity-plugin-for-claude-code
- Estratégia de render pipelines da Unity para 2026: https://unity.com/topics/render-pipelines-strategy-for-2026
- Unity Perception (descontinuado): https://github.com/Unity-Technologies/com.unity.perception | Fork Unity 6: https://github.com/tobeornottobia/com.unity.perception
- EasySynth: https://github.com/ydrive/EasySynth | UnrealCV: https://github.com/unrealcv/unrealcv | Cosys-AirSim: https://github.com/Cosys-Lab/Cosys-AirSim
- Project AirSim (IAMAI, MIT): https://github.com/iamaisim/ProjectAirSim
- Google Map Tiles API Policies: https://developers.google.com/maps/documentation/tile/policies
- Fab EULA: https://www.fab.com/eula
- Agisoft De-Lighter e Unity DeLightingTool: https://github.com/Unity-Technologies/DeLightingTool
- 3DCodeBench: https://arxiv.org/abs/2606.01057 | LL3M: https://arxiv.org/abs/2508.08228 | Thinking in Blender: https://arxiv.org/pdf/2606.02580
- Brush (3DGS em Metal, sem CUDA): https://github.com/ArthurBrussee/brush
- Diorama de favela em UE5, breakdown: https://80.lv/articles/making-a-realistic-favela-diorama-with-maya-substance-3d-ue5

Levantamento produzido em 12 de setembro de 2026. Versões de software, preços e disponibilidade verificados nessa data e sujeitos a mudança.